Build Task — Retriever Module — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python — how a real codebase would actually write it).

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three small files that work together: one that splits text into chunks, one that builds a searchable store out of those chunks, and one that searches it. Every piece was practiced in this document's exercises:

- `chunking.py` — `chunk_by_chars()` and `chunk_by_paragraph()`, copied from the Intermediate exercise. Each is its own named function, so you can swap one for the other.
- `ingest.py` — `build_vector_store(doc_folder, chunk_fn, collection_name)`: the Real-world exercise's Basic Approach 2 — `os.listdir()`, `.endswith(".txt")`, `os.path.join()`, `with open(...)` — but it chunks each file and stores `source_file` **and** `chunk_index` as metadata. All chunks are embedded in one call (`input=` a list).
- `retriever.py` — `retrieve(collection, query, k=3)`: embed the question, `collection.query(...)`, and turn `results["documents"]`, `results["metadatas"]` and `results["distances"]` into a list of dicts. If `collection.count() == 0`, return `[]` before searching.
- `config.py`, `exceptions.py`, `logging_setup.py` — copied from Doc01's Build Task. `config.py` gets one new field, `embedding_model`, read with `os.getenv("EMBEDDING_MODEL", "text-embedding-3-small")` — the same "optional, with a default" pattern as `LOG_LEVEL`.
- `test_retriever.py` — the 4 Test Cases, one function each, in the plain `case` print style of Doc04–07.
- `docs/` — copy the 6 coffee files from the Real-world exercise's `docs/` folder.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

The same files, written the way the Intermediate approaches of the exercises wrote them:

- **`Path(folder).glob("*")` and `path.suffix`** — the Real-world exercise's Approach 5, to keep only `.txt` / `.md` files.
- **`chunk_fn: Callable[[str], list[str]]`** — the Intermediate exercise's Approach 4: the chunker is a parameter, so `build_vector_store(folder, chunk_by_chars)` swaps the method without editing `ingest.py`.
- **One id per chunk, built from the file and position:** `f"{path.name}-{chunk_index}"`. The same file always gives the same ids, never random ones.
- **Embeddings in batches** — the Failure exercise's Approach 2: one call per 1,000 texts, never one call per chunk.
- **A clean rebuild** — the Real-world exercise's Approach 6: `delete_collection()`, catching `chromadb.errors.NotFoundError` on the first run, then `create_collection()`.
- **LangChain's `Chroma`** — the Real-world exercise's Approach 7: `OpenAIEmbeddings(model=...)`, `Chroma(collection_name=..., persist_directory=...)`, `delete_collection()`, `Chroma.from_texts(...)` and `similarity_search_with_score()` (each result has `.page_content` and `.metadata`).
- **`test_retriever.py` with PASS / FAIL** — each case checks its own result, instead of you reading every line.

Think past "ingest once and it works," too. Ask: **what happens if `build_vector_store()` runs a second time on the same folder?** With a plain `create_collection()`, it crashes — the collection already exists. And `collection.add()` with an id that's already stored is silently ignored, so an edited file would keep its old text. The answer is the clean rebuild: delete the collection and build it again from the files as they are now.

A second question: what happens when `doc_folder` has thousands of files? One embedding call per chunk is slow. Collect the chunks from *all* files first, then embed them in a few large batches.

Difference between Basic and Intermediate: Basic names each file's job and the exercise each piece comes from. Intermediate adds the pieces that matter once ingest runs more than once, or on more than a handful of files — stable ids, batches, a clean rebuild — plus a LangChain version.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
chunking.py:
    function chunk_by_chars(text, chunk_size=300):
        step through text in windows of chunk_size characters
        return the list of pieces

    function chunk_by_paragraph(text):
        split text on blank lines
        strip each piece, drop empty ones
        return the list of pieces

ingest.py:
    function build_vector_store(doc_folder, chunk_fn=chunk_by_paragraph,
                                collection_name="docs"):
        config = load_config(), logger = get_logger(__name__)
        make a new empty collection named collection_name
        for each .txt or .md file in doc_folder:
            read its text
            chunks = chunk_fn(text)
            for each chunk, remember its index in this file's chunk list
        if there are no chunks: log a warning, return the empty collection
        embed all the chunks in one call, with config.embedding_model
        add them to the collection, with source_file and chunk_index
        log how many chunks were ingested
        return the collection

retriever.py:
    function retrieve(collection, query, k=3):
        if the collection is empty: return []
        embed the query with the same model
        search the collection for the k closest chunks
        build a dict for each result:
            text, source_file, chunk_index, score
        log the query and how many results came back
        return the list of dicts
```

The trickiest part — tracking which file and which chunk-index each piece of text came from, so it survives all the way into the search results:

```
def build_vector_store(doc_folder, chunk_fn):
    all_texts = []
    all_metadatas = []
    all_ids = []

    for file_name in os.listdir(doc_folder):
        if not (file_name.endswith(".txt") or file_name.endswith(".md")):
            continue
        path = os.path.join(doc_folder, file_name)
        with open(path, "r") as f:
            text = f.read()

        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": file_name, "chunk_index": chunk_index,
            })
            all_ids.append(file_name + "-" + str(chunk_index))

    # your turn: embed all_texts in one call, then collection.add(...)
```

`all_texts[i]`, `all_metadatas[i]` and `all_ids[i]` always describe the same chunk — keep the three lists in step.

And the empty-store guard for `retrieve()`:

```
def retrieve(collection, query, k=3):
    if collection.count() == 0:
        return []
    # ... otherwise search normally
```

Try finishing the rest yourself before looking at the Solution.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

`chunking.py` — the same two functions, with type hints.

`ingest.py`:

```
function load_chunks(doc_folder, chunk_fn) -> (texts, metadatas, ids):
    for each path in Path(doc_folder).glob("*"):
        skip it unless path.suffix is ".txt" or ".md"
        chunks = chunk_fn(path.read_text())
        for chunk_index, chunk_text in enumerate(chunks):
            add chunk_text, {source_file, chunk_index},
                and the id f"{path.name}-{chunk_index}"
    return the three lists

function build_vector_store(doc_folder, chunk_fn=chunk_by_paragraph,
                            collection_name="docs"):
    delete the old collection (NotFoundError on the first run is fine)
    create it again
    texts, metadatas, ids = load_chunks(doc_folder, chunk_fn)
    if no texts: return the empty collection
    embed the texts in batches of 1,000
    add everything in one collection.add(...) call
    return the collection
```

`retriever.py` — the Basic plan, with type hints and `zip()` over the three result lists.

One piece to get you unstuck — collecting the chunks, with their metadata and stable ids:

```
def load_chunks(
    doc_folder: str, chunk_fn: Callable[[str], list[str]],
) -> tuple[list[str], list[dict], list[str]]:
    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in DOC_SUFFIXES:
            continue
        # your turn: chunk the file's text with chunk_fn, then append
        # each chunk's text, its metadata dict and its id
```

```
        ...
    return all_texts, all_metadatas, all_ids
```

The empty-store check has to happen *before* you search — searching an empty store is exactly the case the requirements call out as "must not crash."

Then sketch the clean rebuild and the batched embedding call yourself — both are in the exercises (Real-world Approach 6, Failure Approach 2) — before checking the Solution.

Difference between Basic and Intermediate: Basic builds a working ingest-and-search pipeline for a clean, one-time run. Intermediate adds what matters once this runs more than once, or on more than a handful of files — stable ids, a clean rebuild, and batched embedding calls.

Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Read both depths below — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Your files:

```
08_rag/practice/build_task/
├── docs/                the 6 coffee files from the Real-world exercise
├── empty_docs/          made by test_retriever.py (case 4)
├── chunking.py          chunk_by_chars(), chunk_by_paragraph()
├── config.py            Doc01's file + embedding_model
├── exceptions.py        Doc01's file, unchanged
├── ingest.py            build_vector_store()
├── logging_setup.py     Doc01's file, unchanged
├── retriever.py         retrieve(collection, query, k)
└── test_retriever.py    the 4 Test Cases
```

Copy `exceptions.py` and `logging_setup.py` from Doc01's `practice/build_task/` unchanged. Put your `OPENAI_API_KEY` in a `.env` file in this folder (Doc01).

Basic Version

Approach 1 — raw `chromadb`, with Doc01's config and logger

Story — `chunking.py`: the Intermediate exercise's two chunkers, copied as they are. **If not:** "swap the chunker" would mean rewriting `ingest.py`.

```
# chunking.py

def chunk_by_chars(text, chunk_size=300):
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i + chunk_size])
    return chunks


def chunk_by_paragraph(text):
    chunks = []
    for piece in text.split("\n\n"):
```

```
        stripped = piece.strip()
        if stripped != "":
            chunks.append(stripped)
    return chunks
```

Story — `config.py` : Doc01's file, plus one optional setting, `embedding_model`. `ingest.py` and `retriever.py` must use the **same** model — reading it from one place makes that automatic. (Doc01's comments are left out here to keep it short.) **If not:** changing the model in one file but not the other would compare vectors from two different models, and every search would quietly return nonsense.

```
# config.py — Doc01's file, plus the embedding_model setting
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str
    embedding_model: str   # new in Doc08


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    # new in Doc08: optional — the default model if unset
    embedding_model = os.getenv("EMBEDDING_MODEL", "text-embedding-3-small")
    return Config(
        openai_api_key=api_key,
        log_level=log_level,
        embedding_model=embedding_model,
    )
```

Story — `ingest.py` : the Real-world exercise's Basic Approach 2, now chunking each file and storing `source_file` and `chunk_index` for every chunk, with one embedding call for all chunks. **If not:** a search result couldn't say which file, and which part of it, it came from.

```
# ingest.py
import os
import chromadb
from openai import OpenAI
from chunking import chunk_by_paragraph
from config import load_config
from logging_setup import get_logger

chroma_client = chromadb.Client()


def build_vector_store(
    doc_folder, chunk_fn=chunk_by_paragraph, collection_name="docs",
):
    config = load_config()
    logger = get_logger(__name__)
    client = OpenAI()
    collection = chroma_client.create_collection(name=collection_name)

    all_texts = []
    all_metadatas = []
    all_ids = []

    for file_name in os.listdir(doc_folder):
        if not (file_name.endswith(".txt") or file_name.endswith(".md")):
            continue
        path = os.path.join(doc_folder, file_name)
        with open(path, "r") as f:
            text = f.read()

        chunks = chunk_fn(text)
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": file_name, "chunk_index": chunk_index,
            })
            all_ids.append(file_name + "-" + str(chunk_index))

    if len(all_texts) == 0:
        logger.warning(f"no documents found in {doc_folder}")
        return collection

    response = client.embeddings.create(
```

```
        model=config.embedding_model, input=all_texts,
    )
    embeddings = []
    for item in response.data:
        embeddings.append(item.embedding)

    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    logger.info(f"ingested {len(all_texts)} chunks from {doc_folder}")
    return collection
```

Story — `retriever.py`: the one function Project 3 calls — it checks for an empty store first, then returns plain dicts with a fixed set of keys. **If not:** an empty store would crash the caller, and every caller would have to dig through Chroma's nested result lists itself.

```
# retriever.py
from openai import OpenAI
from config import load_config
from logging_setup import get_logger


def retrieve(collection, query, k=3):
    logger = get_logger(__name__)
    if collection.count() == 0:
        logger.info(f"empty store, no results for: {query}")
        return []

    config = load_config()
    client = OpenAI()
    response = client.embeddings.create(
        model=config.embedding_model, input=[query],
    )
    query_embedding = response.data[0].embedding

    results = collection.query(query_embeddings=[query_embedding], n_results=k)

    documents = []
    texts = results["documents"][0]
    metadatas = results["metadatas"][0]
    distances = results["distances"][0]

    for i in range(len(texts)):
        documents.append({
            "text": texts[i],
```

```
            "source_file": metadatas[i]["source_file"],
            "chunk_index": metadatas[i]["chunk_index"],
            "score": distances[i],
        })

    logger.info(f"query {query!r}: {len(documents)} results")
    return documents
```

Story — `test_retriever.py`: the 4 Test Cases, one function each, in the plain `case` style of Doc04–07. **If not:** "the off-topic question scores worse" would be something you checked once by hand.

```
# test_retriever.py
# Runs the 4 Test Cases from the README. Run it from inside this folder:
# python test_retriever.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from pathlib import Path
from ingest import build_vector_store
from retriever import retrieve


def test_word_for_word(collection):
    query = "Store coffee beans in an airtight container, away from light."
    results = retrieve(collection, query)
    print("case 1 (word-for-word) -> #1:", results[0]["source_file"])


def test_reworded(collection):
    results = retrieve(collection, "How long do ground beans stay good?")
    sources = []
    for result in results:
        sources.append(result["source_file"])
    print("case 2 (reworded)      -> top 3:", sources)


def test_off_topic(collection):
    on_topic = retrieve(collection, "Which grind fits a French press?", k=1)
    off_topic = retrieve(collection, "How do I train for a marathon?", k=1)
    print(
        "case 3 (off-topic)     -> best distance",
        round(off_topic[0]["score"], 2),
        "vs on-topic", round(on_topic[0]["score"], 2),
    )


def test_empty_store():
```

```
    Path("empty_docs").mkdir(exist_ok=True)
    empty = build_vector_store("empty_docs", collection_name="empty")
    print("case 4 (empty store)   ->", retrieve(empty, "anything"))


if __name__ == "__main__":
    collection = build_vector_store("docs")
    test_word_for_word(collection)
    test_reworded(collection)
    test_off_topic(collection)
    test_empty_store()
```

Expected output (`LOG_LEVEL=WARNING` in `.env` hides the INFO lines; your distances will differ):

```
... ingest WARNING no documents found in empty_docs
case 1 (word-for-word) -> #1: Storing coffee beans.txt
case 2 (reworded)      -> top 3: ['Storing coffee beans.txt', ...]
case 3 (off-topic)     -> best distance 1.78 vs on-topic 1.14
case 4 (empty store)   -> []
```

Case 3's score is a distance: the off-topic question's *best* match is still farther away than the on-topic question's best match.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints, `pathlib`, a swappable `chunk_fn`, PASS / FAIL tests

Story — `ingest.py` (Intermediate): `load_chunks()` reads and chunks the files (the Real-world exercise's Approach 5), and `chunk_fn` is typed as `Callable[[str], list[str]]` (the Intermediate exercise's Approach 4). Every id is built from the file and position. **If not:** reading files and talking to Chroma would be tangled in one long function, and nothing would say what kind of function `chunk_fn` must be.

`chunking.py` gets type hints (`text: str` , `-> list[str]`); nothing else changes.

```
# ingest.py
from pathlib import Path
from typing import Callable

import chromadb
from openai import OpenAI

from chunking import chunk_by_paragraph
from config import load_config
from logging_setup import get_logger

DOC_SUFFIXES = (".txt", ".md")

chroma_client = chromadb.Client()


def load_chunks(
    doc_folder: str, chunk_fn: Callable[[str], list[str]],
) -> tuple[list[str], list[dict], list[str]]:
    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in DOC_SUFFIXES:
            continue
        chunks = chunk_fn(path.read_text())
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
```

```
            })
            # why: the same file and position always give the same id
            all_ids.append(f"{path.name}-{chunk_index}")
    return all_texts, all_metadatas, all_ids


def build_vector_store(
    doc_folder: str,
    chunk_fn: Callable[[str], list[str]] = chunk_by_paragraph,
    collection_name: str = "docs",
):
    config = load_config()
    logger = get_logger(__name__)
    collection = chroma_client.create_collection(name=collection_name)

    all_texts, all_metadatas, all_ids = load_chunks(doc_folder, chunk_fn)
    if not all_texts:
        logger.warning(f"no documents found in {doc_folder}")
        return collection

    client = OpenAI()
    response = client.embeddings.create(
        model=config.embedding_model, input=all_texts,
    )
    embeddings: list[list[float]] = []
    for item in response.data:
        embeddings.append(item.embedding)

    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    logger.info(f"ingested {len(all_texts)} chunks from {doc_folder}")
    return collection
```

```
# retriever.py
from openai import OpenAI

from config import load_config
from logging_setup import get_logger


def retrieve(collection, query: str, k: int = 3) -> list[dict]:
    logger = get_logger(__name__)
    if collection.count() == 0:
        # why: an empty store must return [], not crash (Requirements)
```

```
        logger.info(f"empty store, no results for: {query}")
        return []

    config = load_config()
    client = OpenAI()
    response = client.embeddings.create(
        model=config.embedding_model, input=[query],
    )
    query_embedding = response.data[0].embedding

    results = collection.query(query_embeddings=[query_embedding], n_results=k)
    texts = results["documents"][0]
    metadatas = results["metadatas"][0]
    distances = results["distances"][0]

    documents: list[dict] = []
    for text, metadata, distance in zip(texts, metadatas, distances):
        documents.append({
            "text": text,
            "source_file": metadata["source_file"],
            "chunk_index": metadata["chunk_index"],
            "score": distance,
        })

    logger.info(f"query {query!r}: {len(documents)} results")
    return documents
```

Story — `test_retriever.py` (Intermediate): each case now checks its own result and prints PASS or FAIL. **If not:** you'd have to read every printed line and decide yourself whether it's right.

```
# test_retriever.py
# Runs the 4 Test Cases from the README and prints PASS or FAIL for each.
# Run it from inside this folder: python test_retriever.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from pathlib import Path

from ingest import build_vector_store
from retriever import retrieve


def report(name: str, passed: bool, detail: str) -> None:
    if passed:
        print(f"PASS  {name} -> {detail}")
    else:
        print(f"FAIL  {name} -> {detail}")
```

```
def test_word_for_word(collection) -> None:
    query = "Store coffee beans in an airtight container, away from light."
    top = retrieve(collection, query)[0]["source_file"]
    report("case 1 (word-for-word)", top == "Storing coffee beans.txt", top)


def test_reworded(collection) -> None:
    results = retrieve(collection, "How long do ground beans stay good?")
    sources: list[str] = []
    for result in results:
        sources.append(result["source_file"])
    passed = "Storing coffee beans.txt" in sources
    report("case 2 (reworded)", passed, f"top 3: {sources}")


def test_off_topic(collection) -> None:
    on_topic = retrieve(collection, "Which grind fits a French press?", k=1)
    off_topic = retrieve(collection, "How do I train for a marathon?", k=1)
    on_score = on_topic[0]["score"]
    off_score = off_topic[0]["score"]
    # how: a distance, so the off-topic best match must be FARTHER away
    passed = off_score > on_score
    detail = f"distance {off_score:.2f} vs on-topic {on_score:.2f}"
    report("case 3 (off-topic)", passed, detail)


def test_empty_store() -> None:
    Path("empty_docs").mkdir(exist_ok=True)
    empty = build_vector_store("empty_docs", collection_name="empty")
    results = retrieve(empty, "anything")
    report("case 4 (empty store)", results == [], str(results))


if __name__ == "__main__":
    collection = build_vector_store("docs")
    test_word_for_word(collection)
    test_reworded(collection)
    test_off_topic(collection)
    test_empty_store()
```

Expected output (your distances will differ):

```
... ingest WARNING no documents found in empty_docs
PASS  case 1 (word-for-word) -> Storing coffee beans.txt
PASS  case 2 (reworded) -> top 3: ['Storing coffee beans.txt', ...]
```

```
PASS  case 3 (off-topic) -> distance 1.78 vs on-topic 1.14
PASS  case 4 (empty store) -> []
```

Difference from Basic: same logic, with full type hints, `pathlib`, a typed `chunk_fn`, and `load_chunks()` as its own function. The tests check themselves.

Approach 2 — a clean rebuild, and embeddings in batches

Story: running Approach 1's `build_vector_store()` a second time crashes — the collection already exists. And `collection.add()` with an id that's already stored is silently ignored: the old text is kept. **If not:** re-running ingest after editing a document would either crash or keep answering from the old text.

Only `ingest.py` changes. `rebuild_collection()` is the Real-world exercise's Approach 6; `embed_in_batches()` is the Failure exercise's Approach 2. `retriever.py` and `test_retriever.py` stay the same.

```
# ingest.py
from pathlib import Path
from typing import Callable

import chromadb
from chromadb.errors import NotFoundError
from openai import OpenAI

from chunking import chunk_by_paragraph
from config import load_config
from logging_setup import get_logger

DOC_SUFFIXES = (".txt", ".md")
BATCH_SIZE = 1000   # one embeddings call accepts up to 2,048 texts

chroma_client = chromadb.Client()


def rebuild_collection(name: str):
    try:
        chroma_client.delete_collection(name=name)
    except NotFoundError:
        pass  # first run: there was nothing to delete
    return chroma_client.create_collection(name=name)


def embed_in_batches(
    client: OpenAI, model: str, texts: list[str],
) -> list[list[float]]:
```

```
    embeddings: list[list[float]] = []
    for start in range(0, len(texts), BATCH_SIZE):
        response = client.embeddings.create(
            model=model, input=texts[start:start + BATCH_SIZE],
        )
        for item in response.data:
            embeddings.append(item.embedding)
    return embeddings


def load_chunks(
    doc_folder: str, chunk_fn: Callable[[str], list[str]],
) -> tuple[list[str], list[dict], list[str]]:
    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    all_ids: list[str] = []
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in DOC_SUFFIXES:
            continue
        chunks = chunk_fn(path.read_text())
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })
            # why: the same file and position always give the same id
            all_ids.append(f"{path.name}-{chunk_index}")
    return all_texts, all_metadatas, all_ids


def build_vector_store(
    doc_folder: str,
    chunk_fn: Callable[[str], list[str]] = chunk_by_paragraph,
    collection_name: str = "docs",
):
    config = load_config()
    logger = get_logger(__name__)
    # why: a clean rebuild, so an edited or deleted file is never stale
    collection = rebuild_collection(collection_name)

    all_texts, all_metadatas, all_ids = load_chunks(doc_folder, chunk_fn)
    if not all_texts:
        logger.warning(f"no documents found in {doc_folder}")
        return collection

    client = OpenAI()
    embeddings = embed_in_batches(client, config.embedding_model, all_texts)
```

```
    collection.add(
        ids=all_ids, embeddings=embeddings,
        documents=all_texts, metadatas=all_metadatas,
    )
    logger.info(f"ingested {len(all_texts)} chunks from {doc_folder}")
    return collection
```

Running `test_retriever.py` gives the same 4 PASS lines — and now `build_vector_store("docs")` can run again and again: the count stays 6, never 12. Always use the collection a rebuild **returns** — an older `collection` variable points at the deleted one.

Approach 3 — LangChain's `Chroma` wrapper, same clean rebuild

Story: the same module, built on the Real-world exercise's Approach 7. `Chroma.from_texts()` embeds the chunks for you, the store is saved in `./chroma_db/`, and `similarity_search_with_score()` returns `(Document, score)` pairs. **If not:** you'd only know the raw `chromadb` way, and LangChain-based code would need its own ingest from scratch.

```
# ingest.py
from pathlib import Path
from typing import Callable

from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

from chunking import chunk_by_paragraph
from config import load_config
from logging_setup import get_logger

DOC_SUFFIXES = (".txt", ".md")


def build_vector_store(
    doc_folder: str,
    chunk_fn: Callable[[str], list[str]] = chunk_by_paragraph,
    collection_name: str = "docs",
    persist_directory: str = "./chroma_db",
) -> Chroma:
    config = load_config()
    logger = get_logger(__name__)
    embeddings = OpenAIEmbeddings(model=config.embedding_model)

    # why: a clean rebuild, the same idea as Approach 2
    old_store = Chroma(
        collection_name=collection_name,
```

```
        embedding_function=embeddings,
        persist_directory=persist_directory,
    )
    old_store.delete_collection()

    all_texts: list[str] = []
    all_metadatas: list[dict] = []
    for path in Path(doc_folder).glob("*"):
        if path.suffix not in DOC_SUFFIXES:
            continue
        chunks = chunk_fn(path.read_text())
        for chunk_index, chunk_text in enumerate(chunks):
            all_texts.append(chunk_text)
            all_metadatas.append({
                "source_file": path.name, "chunk_index": chunk_index,
            })

    if not all_texts:
        logger.warning(f"no documents found in {doc_folder}")
        return Chroma(
            collection_name=collection_name,
            embedding_function=embeddings,
            persist_directory=persist_directory,
        )

    store = Chroma.from_texts(
        all_texts, embeddings,
        metadatas=all_metadatas,
        collection_name=collection_name,
        persist_directory=persist_directory,
    )
    logger.info(f"ingested {len(all_texts)} chunks from {doc_folder}")
    return store
```

```
# retriever.py
from langchain_chroma import Chroma

from logging_setup import get_logger


def retrieve(store: Chroma, query: str, k: int = 3) -> list[dict]:
    logger = get_logger(__name__)
    # how: on an empty store this returns [] by itself -- no crash
    results = store.similarity_search_with_score(query, k=k)

    documents: list[dict] = []
```

```
    for document, score in results:
        documents.append({
            "text": document.page_content,
            "source_file": document.metadata["source_file"],
            "chunk_index": document.metadata["chunk_index"],
            "score": score,
        })

    logger.info(f"query {query!r}: {len(documents)} results")
    return documents
```

`test_retriever.py` is Approach 1's file, unchanged — `retrieve()` returns the same dicts, so the same 4 PASS lines come back. A search on an empty LangChain store returns `[]` by itself, so `retrieve()` needs no `count()` check.

Difference from Approach 1, and between Approaches 2/3: Approach 1 builds a store that works the *first* time. Approaches 2 and 3 make re-running ingest safe (delete-then-rebuild) — the thing that matters once the documents change, which is the normal case for a real knowledge base. Approach 2 also batches the embedding calls; Approach 3 lets LangChain do the embedding and saves the store on disk.

Which one should you actually use?

For Project 3, use Approach 2 — raw `chromadb`, a clean rebuild, and `retrieve()` returning plain dicts. Project 3's `ingest.py` and `retriever.py` follow exactly that shape. Use Approach 3 when the rest of your code is already built on LangChain. Keep the clean rebuild whichever store you use — a `build_vector_store()` that crashes or keeps old text on a second run is a real bug waiting for the first time someone edits a document.